

Project: Multi-Stage CI/CD Pipeline

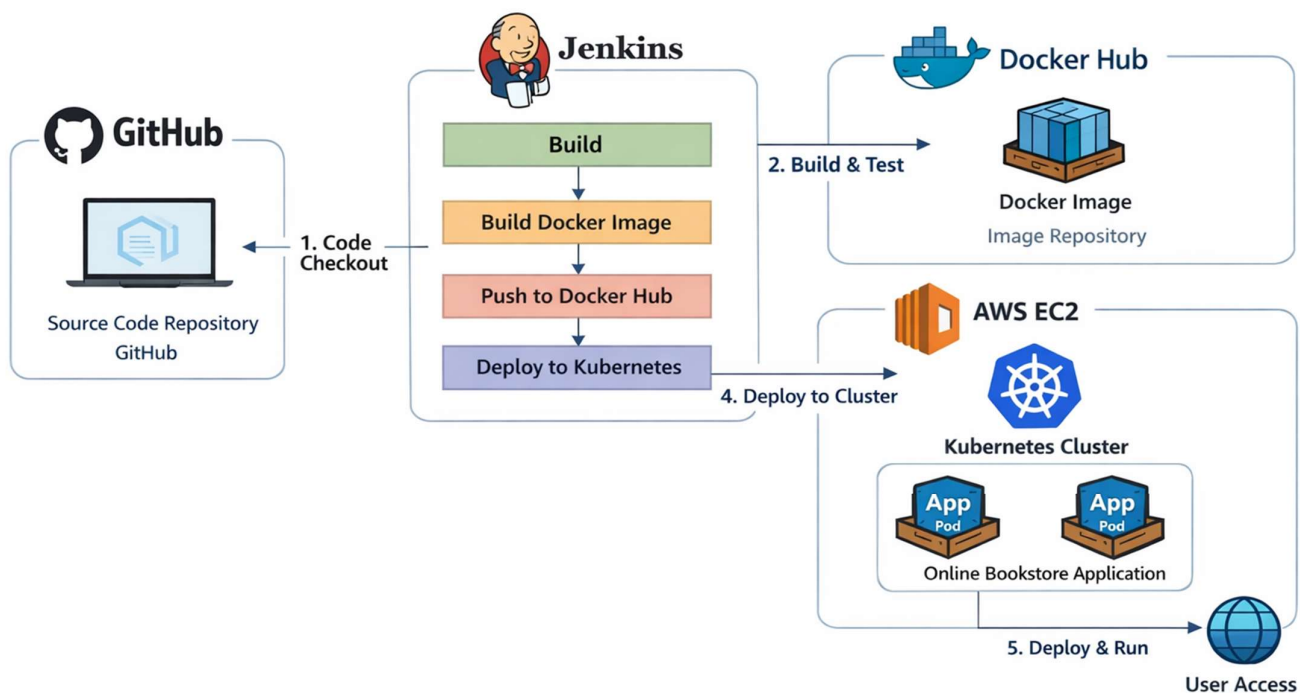
Definition:

A Multi-Stage CI/CD pipeline is an automated software delivery process that integrates code changes, builds the application, creates container images, and deploys the application automatically to a cloud environment. In this project, the pipeline retrieves the source code from GitHub, builds and containerizes the application using Docker, stores the container image in Docker Hub, and deploys the application to a Kubernetes cluster running on a cloud server. The entire workflow is orchestrated by Jenkins, which performs stages such as code checkout, build, Docker image creation, image push, and deployment. The deployment is executed on a Kubernetes cluster hosted on a virtual machine provided by Amazon EC2, where containers are managed and scaled using Kubernetes. This automated pipeline minimizes manual effort, improves deployment reliability, and enables faster and more consistent software delivery.

Scope of the Project:

- Automate the software development lifecycle by implementing a multi-stage CI/CD pipeline.
- Integrate version control using GitHub for managing application source code.
- Implement continuous integration using Jenkins to automatically build and test the application.
- Containerize the application using Docker to ensure consistent deployment across environments.
- Store and manage Docker images in Docker Hub for easy access and version control.
- Deploy the application automatically on a cloud virtual machine provided by Amazon EC2
- Orchestrate containers using Kubernetes to manage deployment, scaling, and availability.
- Reduce manual intervention in the build and deployment process through automation.
- Improve reliability and consistency of application deployments.
- Provide a scalable and efficient DevOps workflow suitable for modern cloud-based applications.

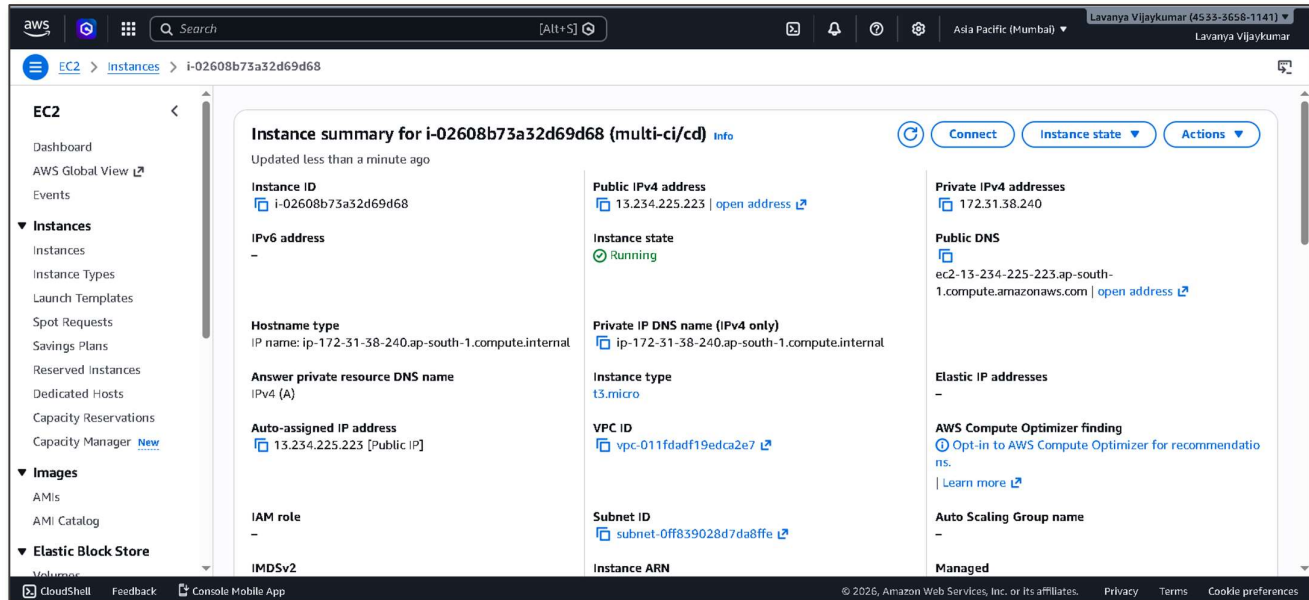
Architecture Diagram:



Project Implementation Steps:

STEP 1: Launch EC2 Instance

- Amazon Linux 2
- t3. micro
- Launch instance.

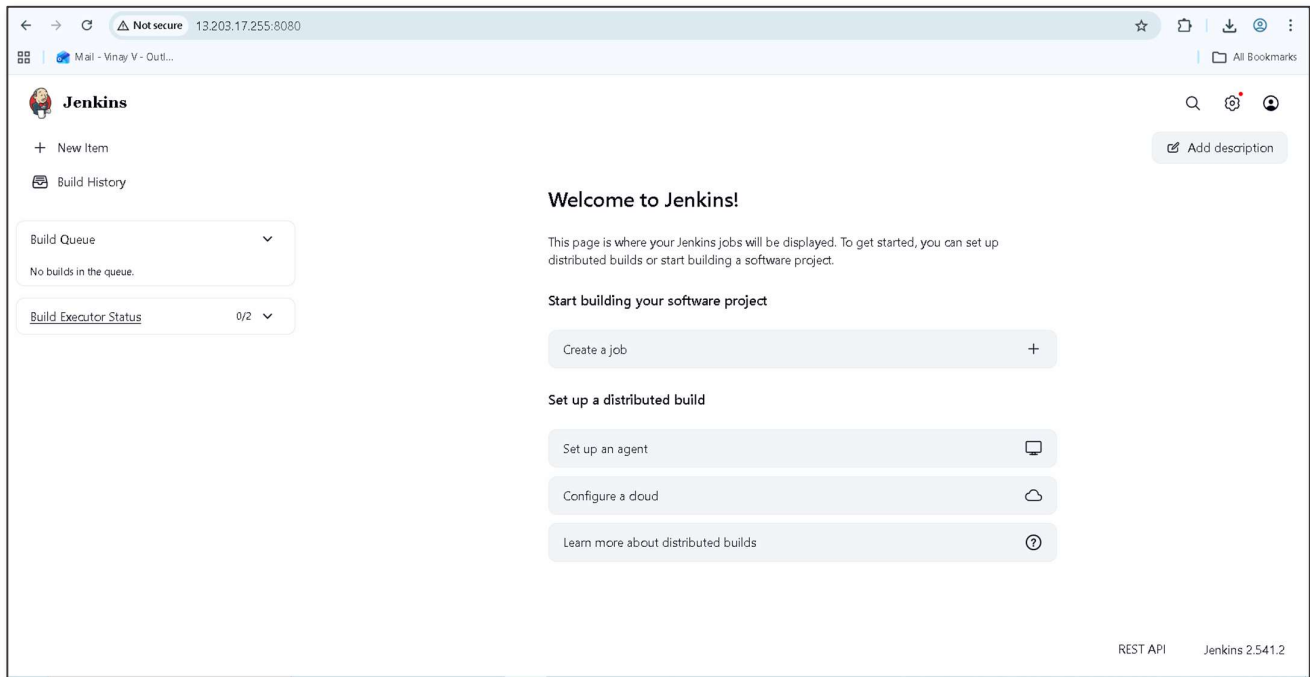


STEP 2: Install Java, Jenkins, docker, maven, git in ec2 instance

```
[ec2-user@ip-172-31-30-177 ~]$ java --version
openjdk 17.0.18 2026-01-20 LTS
OpenJDK Runtime Environment Corretto-17.0.18.9.1 (build 17.0.18+9-LTS)
OpenJDK 64-Bit Server VM Corretto-17.0.18.9.1 (build 17.0.18+9-LTS, mixed mode, sharing)
[ec2-user@ip-172-31-30-177 ~]$ jenkins --version
2.541.2
[ec2-user@ip-172-31-30-177 ~]$ docker --version
Docker version 25.0.14, build 0bab007
[ec2-user@ip-172-31-30-177 ~]$ mvn --version
Apache Maven 3.8.4 (Red Hat 3.8.4-3.amzn2023.0.5)
Maven home: /usr/share/maven
Java version: 17.0.18, vendor: Amazon.com Inc., runtime: /usr/lib/jvm/java-17-amazon-corretto.x86_64
Default locale: en, platform encoding: UTF-8
OS name: "linux", version: "6.1.163-186.299.amzn2023.x86_64", arch: "amd64", family: "unix"
[ec2-user@ip-172-31-30-177 ~]$ git --version
git version 2.50.1
[ec2-user@ip-172-31-30-177 ~]$
```

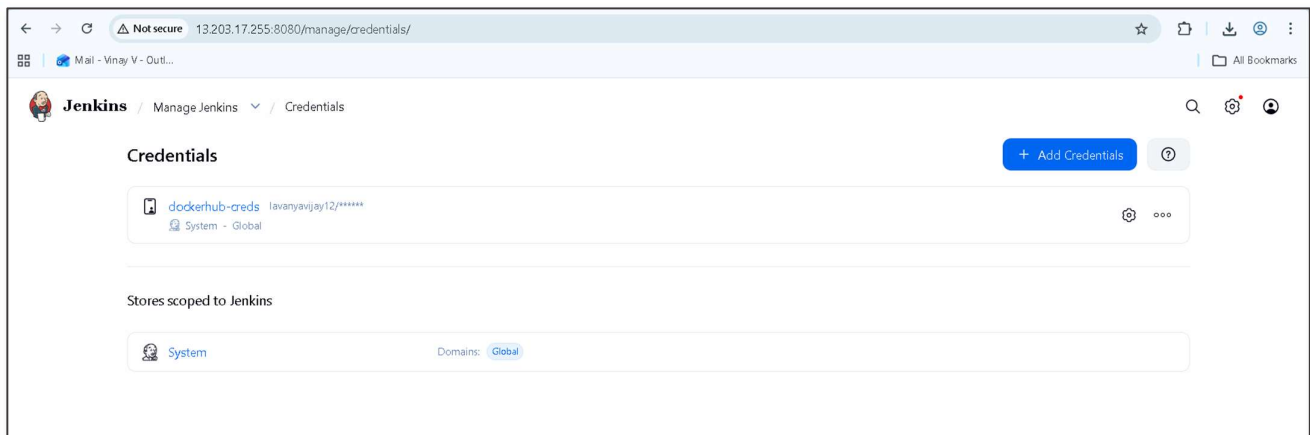
STEP 3: Add custom tcp with port of 8080 in security grp

- Type <http://ec2-ip:8080> URL in browser to access Jenkins dashboard
- Copy paste the InitialAdminPassword
- Install plugins and add username, password etc..



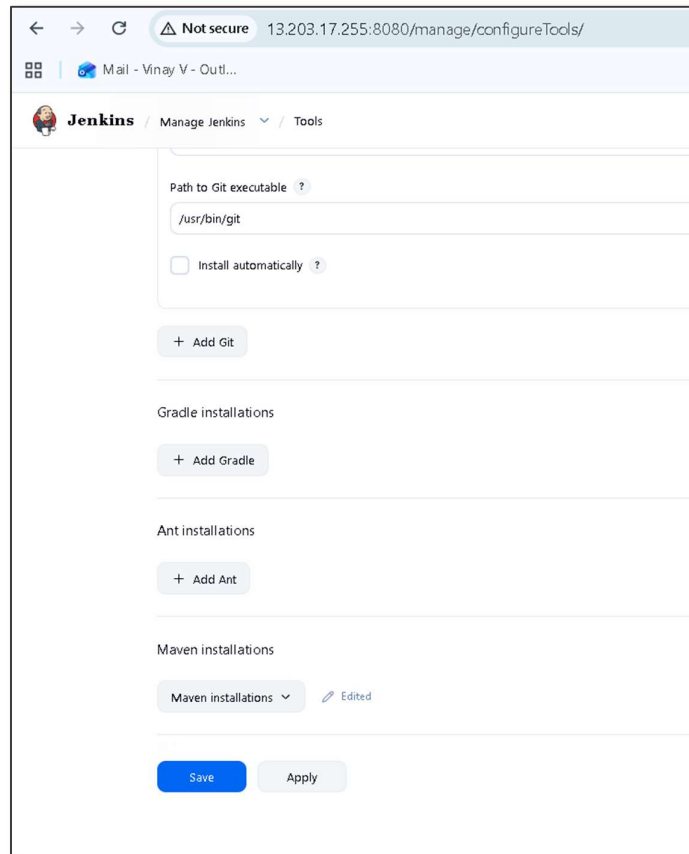
STEP 4: Configure DockerHub Credential

- Manage Jenkins → Credentials → Global → Add Credentials
- Select:
- Kind: Username with password
ID: dockerhub-creds
Username: your DockerHub username
Password: your DockerHub password

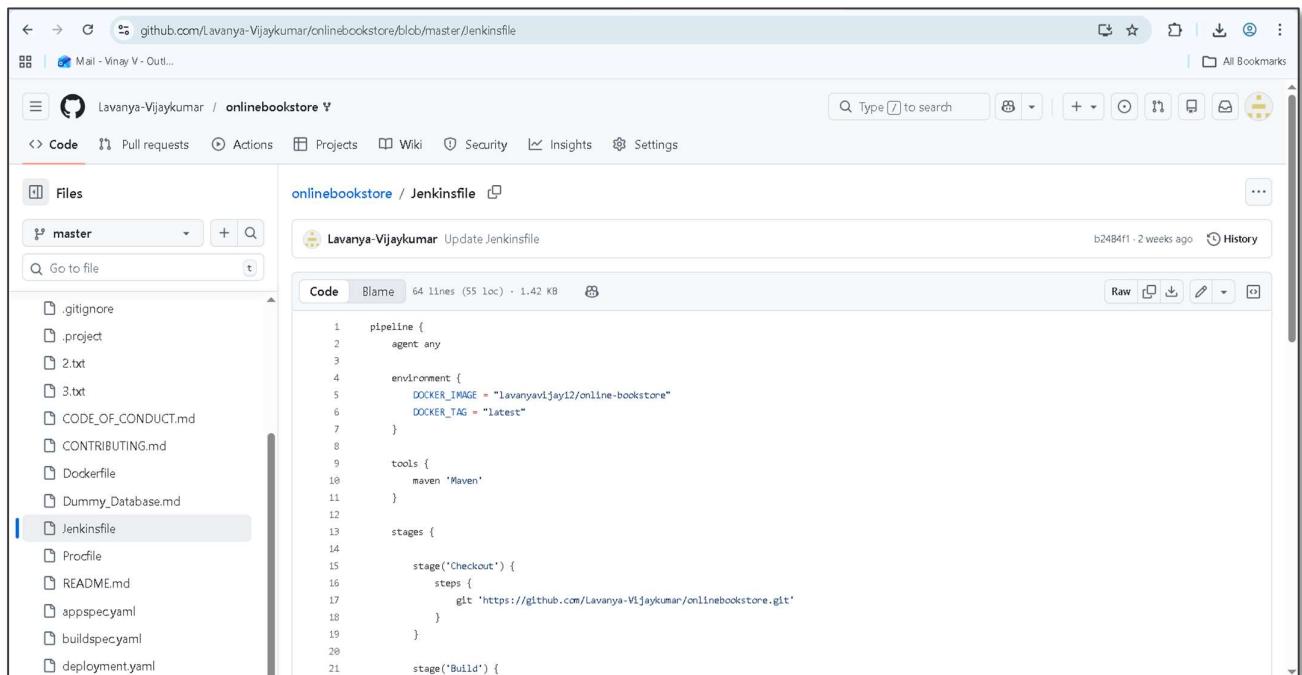


STEP 5: Configure Git and maven in Jenkins UI

- Manage Jenkins → Global Tool Configuration
- Scroll to Git
- Add:Name=Git
- Path to Git:/usr/bin/git
- Scroll to maven
- Add:Name=Maven
- Path to Git:/usr/share/maven
- Click Save

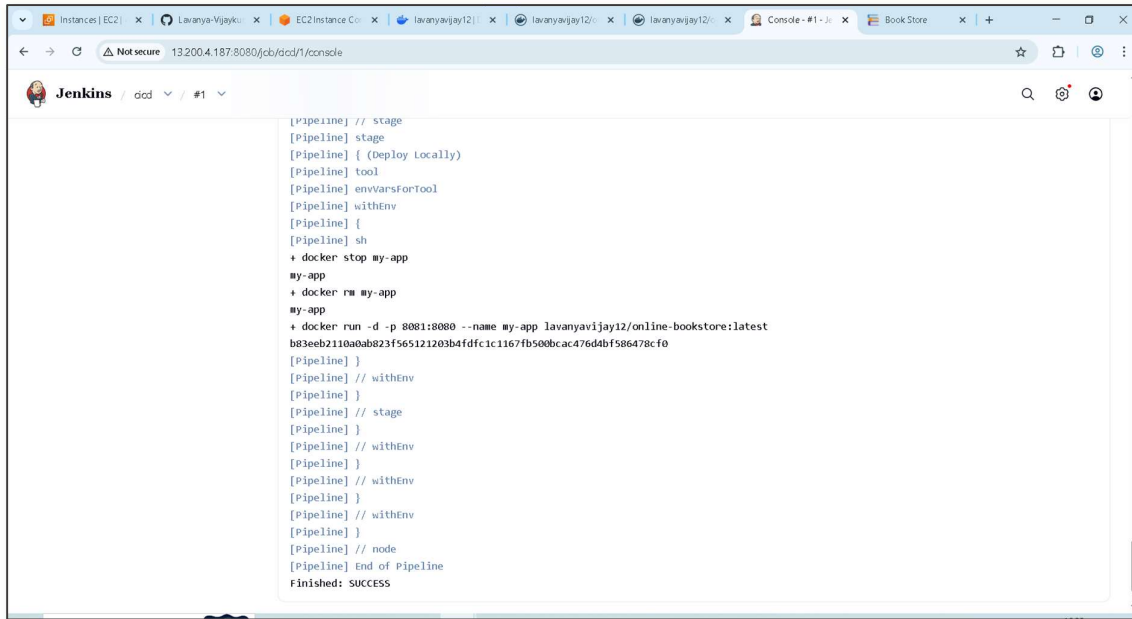


STEP 6: Add jenkinsfiles in git repository



STEP 7: Create Pipeline Job in Jenkins

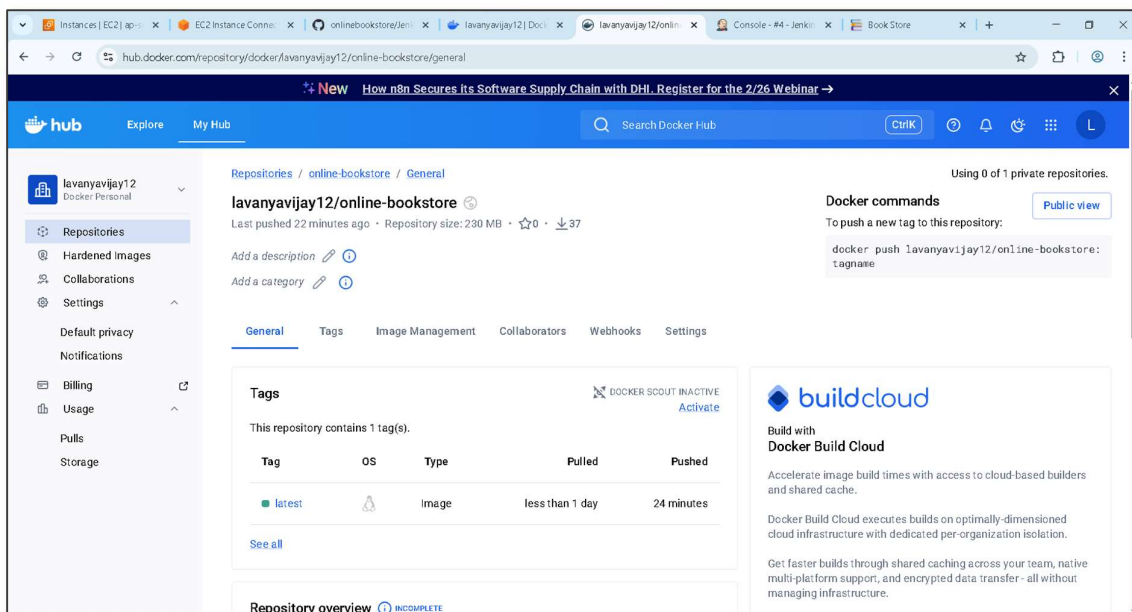
- New Item
- Name: online-bookstore-pipeline
- Select Pipeline
- Pipeline → Definition: Pipeline script from SCM
- SCM: Git
- Repo URL: your GitHub repo
- Branch: main
- Save
- Click: Build Now



```

[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy Locally)
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] withEnv
[Pipeline] {
[Pipeline] sh
+ docker stop my-app
my-app
+ docker rm my-app
my-app
+ docker run -d -p 8081:8080 --name my-app lavanyavijay12/online-bookstore:latest
b03eeb2110a0ab823f565121203b4fdcf1c1167fb500bcac476d4bf586478cf0
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
  
```

STEP 8: Output Build Docker image, Push to DockerHub, Run container in same EC2



hub.docker.com/repository/docker/lavanyavijay12/online-bookstore/general

Repositories / online-bookstore / General

lavanyavijay12/online-bookstore 
 Last pushed 22 minutes ago · Repository size: 230 MB · ☆0 · ↓37

Add a description  
 Add a category  

General Tags Image Management Collaborators Webhooks Settings

Tags  [Activate](#)

This repository contains 1 tag(s).

Tag	OS	Type	Pulled	Pushed
latest		Image	less than 1 day	24 minutes

[See all](#)

Repository overview  INCOMPLETE

Using 0 of 1 private repositories.

Docker commands [Public view](#)

To push a new tag to this repository:

```
docker push lavanyavijay12/online-bookstore:tagname
```

buildcloud

Build with Docker Build Cloud

Accelerate image build times with access to cloud-based builders and shared cache.

Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.

Get faster builds through shared caching across your team, native multi-platform support, and encrypted data transfer - all without managing infrastructure.

```

Installing : perl-lib-0.65-477.amzn2023.0.7.x86_64
Installing : perl-TermReadKey-2.38-9.amzn2023.0.2.x86_64
Installing : perl-File-Find-1.37-477.amzn2023.0.7.noarch
Installing : perl-Error-1.10.17029-5.amzn2023.0.2.noarch
Installing : perl-Git-2.50.1-1.amzn2023.0.1.noarch
Installing : git-2.50.1-1.amzn2023.0.1.x86_64
Running scriptlet: git-2.50.1-1.amzn2023.0.1.x86_64
Verifying : git-2.50.1-1.amzn2023.0.1.x86_64
Verifying : git-core-doc-2.50.1-1.amzn2023.0.1.x86_64
Verifying : git-core-doc-2.50.1-1.amzn2023.0.1.noarch
Verifying : perl-Error-1.10.17029-5.amzn2023.0.2.noarch
Verifying : perl-File-Find-1.37-477.amzn2023.0.7.noarch
Verifying : perl-Git-2.50.1-1.amzn2023.0.1.noarch
Verifying : perl-TermReadKey-2.38-9.amzn2023.0.2.x86_64
Verifying : perl-lib-0.65-477.amzn2023.0.7.x86_64

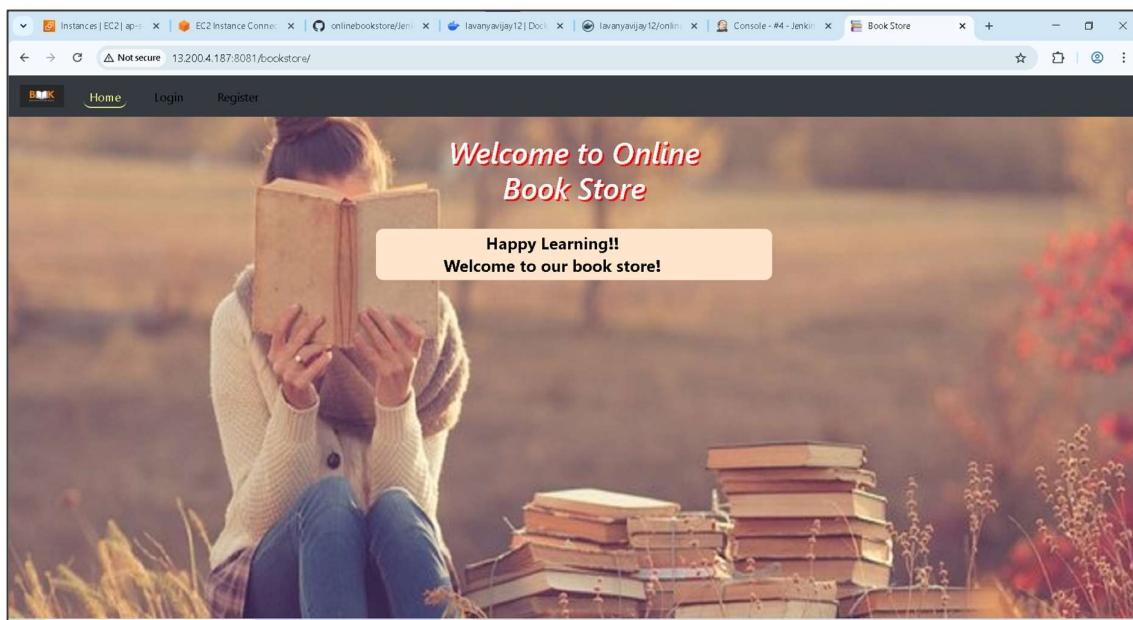
Installed:
git-2.50.1-1.amzn2023.0.1.x86_64      git-core-2.50.1-1.amzn2023.0.1.x86_64      git-core-doc-2.50.1-1.amzn2023.0.1.noarch
perl-Error-1.10.17029-5.amzn2023.0.2.noarch  perl-File-Find-1.37-477.amzn2023.0.7.noarch  perl-Git-2.50.1-1.amzn2023.0.1.noarch
perl-TermReadKey-2.38-9.amzn2023.0.2.x86_64  perl-lib-0.65-477.amzn2023.0.7.x86_64

Complete!
[ec2-user@ip-172-31-17-0 ~]$ which git
/usr/bin/git
[ec2-user@ip-172-31-17-0 ~]$ sudo systemctl restart jenkins
[ec2-user@ip-172-31-17-0 ~]$ sudo systemctl restart jenkins
[ec2-user@ip-172-31-17-0 ~]$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
9414de49c979  lavanyavijay12/online-bookstore:latest  "catalina.sh run"       42 seconds ago  Up 41 seconds  0.0.0.0:8081->8080/tcp, :::8081->8080/tcp  my-app
root@9414de49c979:/usr/local/tomcat# ls
bin      conf      filtered-KEYS  LICENSE  native-jni-lib  README.md  RUNNING.txt  upstream-KEYS  webapps.dist
BUILDING.txt  CONTRIBUTING.md  lib          NOTICE  RELEASE-NOTES  NOTICE
root@9414de49c979:/usr/local/tomcat#

```

STEP 9: Access Application

- Type URL http://YOUR_EC2_IP:8080/bookstore



Conclusion:

The Multi-Stage CI/CD Pipeline project successfully demonstrates how modern DevOps practices can automate the complete software delivery lifecycle. By integrating source code management through GitHub, continuous integration and automation using Jenkins, containerization with Docker, and container orchestration through Kubernetes, the project enables efficient and reliable deployment of applications. The application is deployed on a cloud virtual machine provided by Amazon EC2, ensuring scalability and accessibility. This automated pipeline minimizes manual intervention, reduces deployment errors, improves consistency across environments, and accelerates software delivery. Overall, the project highlights the importance of CI/CD pipelines in modern cloud-based application development and provides a strong foundation for implementing scalable and automated DevOps workflows.